Rapport de projet

OCaml Quest: Functional Sorcery

Projet PFA

19 avril 2026

Résumé. OCaml Quest : Functional Sorcery est un RPG pédagogique dans lequel le joueur apprend l'OCaml en résolvant des énigmes et en avançant dans un monde structuré autour de scènes, de dialogues et de combats scriptés. Le projet met en oeuvre une architecture ECS, un système de collisions AABB, une sauvegarde versionnée, ainsi qu'une logique de scénarios pour guider la progression.

1 Présentation du jeu

Le concept du jeu est simple : la syntaxe devient un grimoire. Le joueur incarne un apprenti qui traverse plusieurs lieux – maison, ville, école, bibliothèque et salle de classe – afin d'apprendre progressivement les bases de la programmation fonctionnelle. Chaque zone présente un contexte narratif et un objectif concret : parler à un personnage, franchir une porte, débloquer un événement, ou valider un exercice de code.

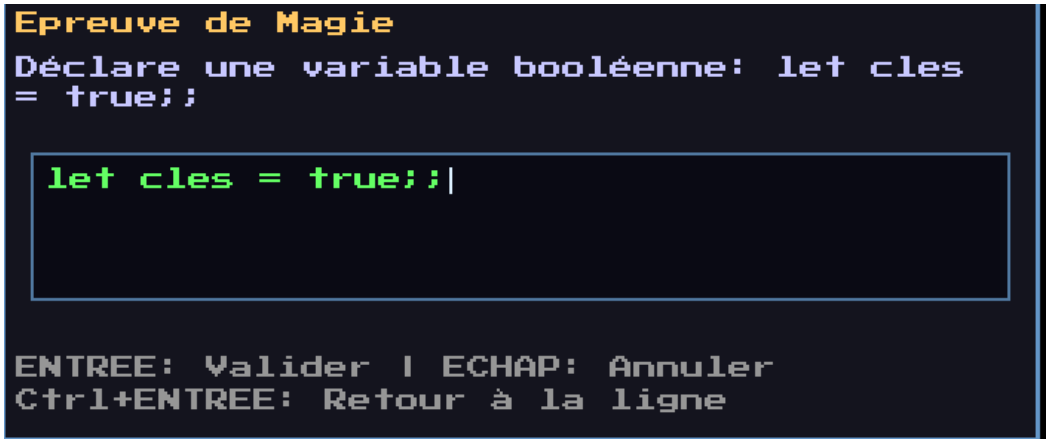

FIGURE 1 – Exemple d'une épreuve de code.

L'identité du projet est clairement pédagogique. Le texte de présentation du dépôt résume bien cette intention : le jeu fait de la magie avec les types, la récursion et le filtrage par motif. Cette logique se retrouve dans la progression des niveaux, qui introduit les notions par paliers plutôt que par un cours théorique abstrait.

1.1 Boucle de jeu et prise en main

Le point d'entrée du jeu est la boucle principale définie dans `src/game.ml`. La structure générale distingue trois états : menu, création de personnage et jeu effectif. Quand la partie est lancée, les systèmes sont mis à jour dans un ordre fixe : événements scénario, déplacement, collisions, transitions de porte, rendu, puis systèmes annexes comme le tutoriel ou les dialogues.

Listing 1 – Boucle principale simplifiee du jeu

```
match global.menu_state with
| Some menu ->
    Input.handle_input (); Menu_system.update (); Menu_system.draw (); Gfx.commit
    global.ctx
| None ->
    Player.stop_player ();
    Input.handle_input ();
    Story_events_system.update ();
    Tutorial_manager.check_scene_tutorials ();
    Move_system.update dt;
    Collision_system.update dt;
    Door_transition_system.update dt;
    Draw_system.update dt;
    Library_cinematic_system.update dt;
    Tutorial_system.update dt;
    Dialogue_system.update dt;
    Code_challenge_system.update dt Seq.empty;
    Library_guide_system.update dt Seq.empty;
```

Le joueur utilise un schéma de contrôles simple : déplacement, interaction et ouverture du menu. Cette sobriété est volontaire, car l'effort cognitif doit porter sur les concepts OCaml et non sur la maîtrise d'un système complexe de commandes.

1.2 Contrôles et gameplay

Les contrôles principaux sont les suivants :

— **ZQSD** : déplacement du joueur (haut, gauche, bas, droite) ;

— **Espace** : interaction contextuelle (parler à un PNJ, lire un panneau, avancer un dialogue) ;

— **E** : interaction alternative contextuelle (notamment avec PNJ et panneaux selon l'état courant) ;

— **Échap** : ouvrir/fermer le menu pause, ou quitter certains panneaux actifs ;

— **Entrée** : validation dans certains écrans (création du personnage, saisie de code, progression de certaines boîtes de dialogue).

Du point de vue gameplay, la boucle alterne exploration, interactions et séquences scriptées. En exploration libre, le joueur se déplace entre scènes et déclenche des événements via les portes, PNJ et objets. Pendant les séquences narratives (cinématiques de classe, duel, transitions scénarisées), les contrôles sont volontairement restreints afin de conserver une mise en scène stable et d'éviter les conflits avec les déplacements scriptés des PNJ.

Les défis de programmation sont intégrés au flux de jeu : le joueur lit un objectif narratif, saisit une solution OCaml, puis reçoit une validation immédiate. Cette intégration maintient un lien direct entre la progression pédagogique et la progression dans le monde du jeu.

1.3 Progression et scènes

La progression se découpe en scènes distinctes. Le joueur commence par la maison et la création du personnage, puis passe à la ville, à l'école, à la bibliothèque et enfin à la salle de classe. Chaque scène remplit un rôle précis :

— la maison sert d'introduction narrative et de zone d'apprentissage initiale ;

— la ville ouvre la phase d'exploration et de lecture des panneaux ;

— l'école introduit les premiers objectifs et les personnages secondaires ;

— la bibliothèque associe exploration et règles de narration ;

— la salle de classe concentre les séquences les plus scriptées, dont le duel final.

Le jeu utilise aussi des panneaux, des portes et des objets interactifs pour rythmer l'avancement. Cette structure permet de guider le joueur sans recours à un tutoriel trop invasif.

2 Architecture et types principaux

Le projet est construit autour d'une approche ECS (Entity-Component-System). Les entités portent l'identité d'un objet du monde, les composants stockent les données, et les systèmes contiennent la logique. Cette séparation rend le code plus modulaire et facilite l'ajout d'un nouveau comportement sans alourdir les classes existantes.

2.1 Types centraux

Le fichier `src/components/component_defs.ml` rassemble les types et classes de base. On y trouve les structures de dialogue, les données de PNJ, les portes, ainsi que les types abstraits qui définissent ce qu'est une entité collidable, drawable ou mobile.

Listing 2 – Types de base du projet

```
type npc_data = {
  name : string;
  dialogue : dialogue;
  scene : Scene.scene;
}

type door_config = {
  id: string;
  current_scene: Scene.scene;
  target_scene: Scene.scene;
  player_spawn_x: int;
  player_spawn_y: int;
}

type tag +=
  | No_tag
  | Player
  | InScene of Scene.scene
  | Door of door_config
```

Dans la pratique, un joueur ou un PNJ est une entité qui possède des composants de position, de boîte de collision, de texture, de vitesse et d'étiquette. Les classes `player`, `npc_component`, `sign_component` et `door` permettent de factoriser les comportements communs tout en gardant des objets simples à manipuler dans les systèmes.

2.2 État global et scènes

L'état global, défini dans `src/core/global.ml`, centralise les ressources de la partie : fenêtre, contexte graphique, joueur, états de dialogues, tutoriel, code challenge, ainsi que les indicateurs de progression. Ce choix est pragmatique : le moteur conserve un point d'accès unique pour l'état courant, ce qui simplifie le pilotage des systèmes.

Le module `src/core/scene.ml` énumère les scènes du jeu. Cette énumération sert de clé de routage pour le rendu, les collisions, les transitions et la logique des événements. En pratique, tout comportement dépend de la scène active, ce qui évite les effets de bord entre zones.

2.3 Monde et initialisation

Le démarrage du monde est rassemblé dans `src/core/world_bootstrap.ml`. Cette étape installe les murs, les portes, les PNJ et les objets interactifs dans les scènes correspondantes. Le

résultat est un monde cohérent dès le lancement, sans initialisation dispersée dans plusieurs systèmes.

3 Point technique : mouvements scriptés et collisions

Le point technique le plus délicat du projet est la gestion des séquences scriptées. Il fallait que les personnages non joueurs puissent se déplacer de façon narrative sans diagonales absurdes, sans rotation continue, et sans pousser le joueur durant les cinématiques. La solution adoptée combine trois mécanismes : routes par étapes, collisions scène-spécifiques, et gel partiel du gameplay pendant les scènes scriptées.

3.1 Gestion des routes

Le système des événements de scénario, dans `src/systems/story_events_system.ml`, orchestre les déplacements des PNJ. Les états internes comme `aerin_exit_started`, `students_exit_started` ou `lambda_return_started` servent à découper une séquence en phases lisibles.

Listing 3 – Choix du mode de navigation

```
let route_mode_for_current_state () =
  let global = Global.get () in
  if
    (Scene.current () = Scene.School && not global.school_students_event_completed)
    ||
    (Scene.current () = Scene.Classroom
     &&
    (not global.classroom_intro_completed
     || !classroom_post_success_started
     || !aerin_exit_started
     || !students_exit_started
     || !lambda_return_started))
  then
    Navigation_debug.Scripted
  else
    Navigation_debug.Dynamic
```

Cette logique permet de basculer entre un mode scripté, utile pour les scènes narratives, et un mode dynamique, réservé aux déplacements ordinaires. Le joueur conserve ainsi une impression de naturel dans l'exploration, tout en laissant le scénario imposer des trajectoires précises quand cela est nécessaire.

3.2 Collisions et affichage

Le rendu des collisions de debug a aussi été généralisé. Le module `src/systems/draw.ml` sélectionne maintenant les rectangles de collision selon la scène active. Concrètement, la vérification des collisions repose sur des calques de collision par scène (maison, ville, école, bibliothèque, classe), ce qui permet de visualiser et de valider rapidement les zones bloquantes. Ce choix est important pour le debug, car il rend visibles les zones bloquées de la maison, de la ville, de l'école, de la bibliothèque et de la classe sans dupliquer du code spécifique au tableau de classe.

Listing 4 – Selection des collisions selon la scene

```
let collision_rects_for_scene =
  match current_scene with
  | Scene.House -> House_map.collision_rects ()
  | Scene.Town -> Town_map.collision_rects ()
  | Scene.School -> School_map.collision_rects ()
  | Scene.Library -> Library_map.collision_rects ()
```

```
  | Scene.Classroom -> Classroom_map.collision_rects ()
  | _ -> []
in
if Cst.debug_draw_classroom_collisions && collision_rects_for_scene <> [] then begin
  List.iter (fun (x, y, w, h) -> ... ) collision_rects_for_scene
end
```

FIGURE 2 – Calque de collisions de la maison du joueur.

FIGURE 3 – Calque de collisions de la ville.

FIGURE 4 – Calque de collisions de l'école.

FIGURE 5 – Calque de collisions de la salle de classe.

FIGURE 6 – Calque de collisions de la bibliothèque.

Le même principe de décomposition se retrouve dans les cartes elles-mêmes : chaque scène expose ses rectangles bloquants, ce qui rend la logique plus lisible et plus simple à maintenir. Cette approche a aussi aidé à corriger les problèmes de collision entre le joueur et les PNJ pendant les

cinématiques.

3.3 Sauvegarde versionnée

Le système de sauvegarde a été versionné pour rester compatible avec les anciens fichiers. Le fichier `src/core/savegame.ml` définit maintenant un payload versionné, puis retombe sur un format legacy si besoin. Cela permet de faire évoluer l'état global sans casser les parties déjà enregistrées.

Listing 5 – Sauvegarde versionnée et compatibilité descendante

```
type file_payload =
  | Save_v2 of data

let load () =
  ...
  match loaded with
  | Save_v2 payload -> Some payload
  with _ ->
    (* Backward compatibility with old unversioned saves. *)
    try
      let payload : legacy_data = Marshal.from_channel in_channel in
      Some { ...; classroom_scripted_movement_active = false; ... }
    with _ ->
      None
```

Le point clé ici est la robustesse. Dès que l'état global évolue, un simple enregistrement binaire non versionné devient fragile. La version de sauvegarde fixe un contrat minimal et évite de perdre le travail du joueur.

4 Tests et validation

La validation du projet repose sur trois niveaux complémentaires : la compilation, les tests automatisés via Dune, puis les tests fonctionnels manuels en jeu.

Une cible de tests automatique a été ajoutée dans le projet via `test/test_game.ml` et `test/dune`. Elle s'exécute avec la commande `dune runtest`. Cette suite couvre notamment :

— la présence des marqueurs de scène en classe (points de placement et sortie) ;

— la vérification du calque de collision PNJ (exclusion des cases de bancs) ;

— la présence des bordures de collision dans la maison et la ville ;

— la validation de plusieurs défis de code OCaml (bool, calcul de puissance, dégâts, exercice de type).

Une deuxième suite, `test/test_cinematics.ml`, couvre les régressions de déplacements scriptés en classe. Elle vérifie en particulier :

— le fallback de ciblage lorsqu'un marqueur est absent ;

— la stabilité de l'assignation des cibles PNJ depuis les marqueurs (`1`, `2`, `3`, `A`, `L`) ;

— le caractère axis-aligné des segments de waypoints ;

— le nombre attendu de waypoints par PNJ (garde-fou anti-régression des chorégraphies).

L'exécution de `dune runtest` valide actuellement l'ensemble de ces tests (suite générale et suite cinématique), ce qui fournit une base reproductible pour la partie "Tests" du rapport.

4.1 Validation technique

La première vérification est la compilation avec Dune, en particulier la cible SDL. Cela permet de confirmer que le moteur, les systèmes et les dépendances graphiques s'assemblent correctement. La version web reste aussi disponible via la configuration du dépôt, mais le cœur de la validation se fait sur l'exécutable natif.

Les contrôles manuels ont porté sur les points critiques suivants :

— démarrage du jeu, création de personnage et retour au menu ;

— transition entre les scènes et positionnement du joueur ;

— vérification des collisions via les calques de collision de chaque scène ;

— séquences scriptées de la classe, y compris les déplacements des PNJ ;

— ouverture, chargement et suppression d'une sauvegarde ;

— déclenchement des dialogues et des défis de code.

Pour les collisions, la méthode de test retenue est basée sur les calques : on active l'affichage debug, on affiche les rectangles du calque de la scène courante, puis on vérifie que les collisions réelles correspondent à ces zones. Concrètement, on teste trois cas : passage libre hors calque, blocage strict sur le calque, et stabilité des collisions pendant les mouvements scriptés de PNJ. Cette vérification visuelle permet de détecter rapidement une incohérence entre la carte logique et le comportement en jeu.

4.2 Régressions surveillées

Les régressions les plus sensibles concernaient le déplacement diagonal des PNJ, le comportement de toupie lors des séquences scriptées, le fait de pousser le joueur, et l'affichage de collisions limité à une seule scène. Ces points ont servi de checklist pendant les correctifs successifs.

Une autre vérification importante consistait à recharger une ancienne sauvegarde après les changements de structure. Le chargement compatible devait rester fonctionnel malgré l'ajout de nouveaux champs globaux.

5 Conclusion

OCaml Quest : Functional Sorcery est un projet cohérent avec son objectif pédagogique. Son intérêt principal n'est pas seulement d'illustrer l'OCaml, mais de le transformer en mécanique de jeu. L'architecture ECS, les scènes, les événements scriptés, les collisions par zone et la sauvegarde versionnée forment un ensemble solide, extensible et suffisamment lisible pour supporter de nouvelles leçons.

Le point le plus formateur a été la gestion des séquences narratives : elle a obligé à distinguer proprement le déplacement libre, le déplacement scripté, la collision physique et la présentation visuelle. C'est exactement le genre de séparation qui donne sa robustesse à un moteur de jeu bien structuré.